Asteroids3D

Component-based software engineering project

Student Name: Emil Jørgensen

Student Exam Number: 215661787

GitHub User: Emilioso007

GitHub Repo: https://github.com/Emilioso007/Asteroids3D

Video Demonstration: https://youtu.be/oF3Cr7Afvww

0. Abstract

This paper documents the development of Asteroids3D. The main architectural problem was building a game that could be modified without recompilation, requiring loose coupling between the core engine and individual gameplay elements. This was achieved by combining the Java Platform Module System and the Spring framework to dynamically discover and load gameplay features introduced as separate .jar files. Additionally, an Entity Component System and the Jaylib-ffm library were utilized to handle the game logic and 3D rendering. The end result is a fully functional game where modules can be added or removed without recompiling the core engine.

Contents

1. Introduction

Asteroids3D is a new take on the classical arcade game, where the player is tasked with navigating a spacecraft through a swarm of asteroids, whilst avoiding enemy bullets along the way. As the name suggests, this game is now in three dimensions, which makes the whole process a lot more challenging. You never know what's behind you.

As the player you control roll, pitch and yaw of your spacecraft, as well as the thrusters and guns. Your task is to crack open asteroids and collect their valuable crystals inside. If an enemy comes close, you may also eliminate them to ease your further travel through space.

2. Requirements

The following requirements are needed to achieve both a fun and extensible game.

Functional requirements for the game:

No.	Title	Description
F01	Player	A controllable player entity.
F02	Enemy	Enemy entities shooting at the player.
F03	Asteroid	Asteroids acting as both obstacles and resources.
F04	Bullet	Bullets being shot by both player and enemy.
F05	Crystal	Crystals as the valuable collectible earning the player points.
F06	Physics	The game handles physics in 3D space.
F07	Rendering	The game renders 3D models with various meshes and materials, including LOD support.
F08	Shader	The game uses shaders to mimic lighting and reflectiveness.

Table 1: Functional Requirements

Non-functional requirements for the game:

No.	Title	Description
NF01	Moddable	The game supports loading, updating or removing various components without recompiling the core game.
NF02	Modular	The game uses Java modules to enforce strict encapsulation and prevent tight coupling between components.
NF03	Contracts	The interaction between modules happens through well-defined Service Provider Interfaces, rather than concrete implementations.
NF04	SoC	The game uses a data-oriented ECS architecture, which allows for a great separation of concern between individual systems.
NF05	Microservice	The game provides a microservice for score tally.
NF06	Raylib	The game is using Raylib [1] for rendering.
NF07	JPMS	The game is using JPMS and the ServiceLoader [2] to orchestrate everything.
NF08	Spring	The game uses the Spring framework [3] for dependency injection.
NF09	Test	Unit testing is used to ensure the inner workings of modules stay coherent despite changes in architecture.

Table 2: Non-functional Rerquirements

3. Analysis

The following analysis section describes what the system should do, and what interfaces and entities that could facilitate the requirements stated in section 2. Requirements. This will be documented using various diagrams.

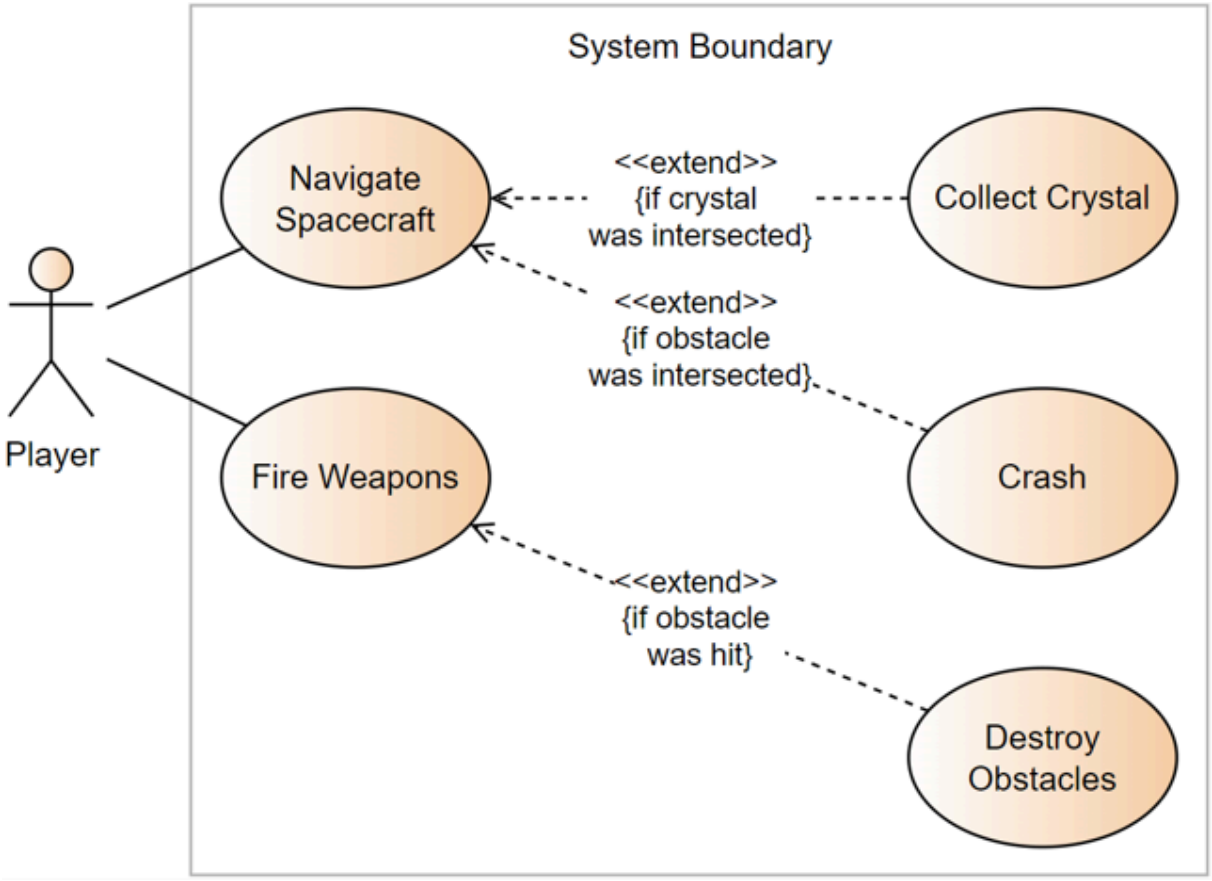

Figure 1: Use Case Diagram that shows what interactions the player can do in the game, as well as what objective each action can lead to.

Based on this diagram (Figure 1) we can see that the system should consist of a player entity that can do various tasks based on user-input.

UC1: When the player navigates the spacecraft, they might intersect a crystal. If so, they collect it and their score is incremented. A high score is the primary goal of the player.

UC2: When the player navigates the spacecraft, they might intersect an obstacle, be it an asteroid, enemy or bullet. This will result in a crash of the spacecraft and game over.

UC3: When the player fires their weapons, they might hit an obstacle, be it an asteroid or enemy. This will destroy said obstacles clearing the path for the player.

This Use Case analysis confirms the proposed entities from the requirements.

- The player is the user-controlled spacecraft.
- The asteroid is an obstacle containing valuable crystals.
- The enemy is an obstacle firing bullets at the player.
- The bullet is a fast-flying entity coming from either the player or the enemies.
- The crystal is the valuable resource.

To link the entities and logic together, some contracts must be provided, namely:

IGamePluginService - Responsible for adding entities to the system.

IEntityProcessorService - Responsible for doing early logic, like movement.

IPostEntityProcessorService - Responsible for doing late logic, like collision detection/resolution.

With these three contracts, it is possible to add entities and perform advanced logic on them.

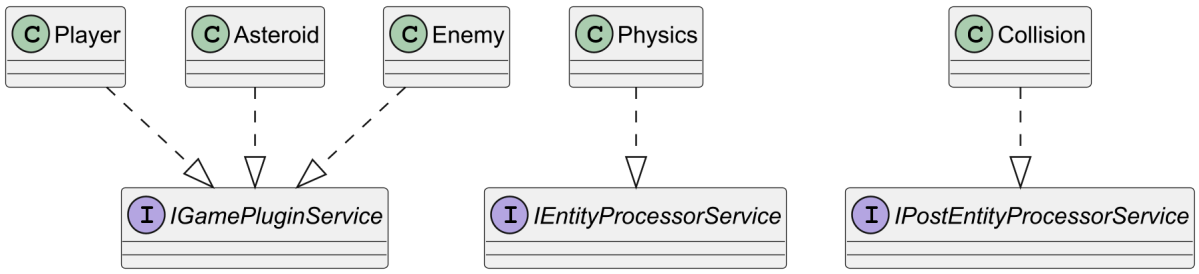

Figure 2: Object Model Diagram showing relation between entities and contracts.

On Figure 2 you can see how these three contracts would be wired up in the system. Bullets and Crystals are both specific edge cases left out for simplicity. The key takeaway is that entities implement the IGamePluginService, telling the system that they want to be added immediately, meanwhile a physics class would implement the IEntityProcessorService, because it needs to calculate movements as the first thing in the frame. A collision class would then implement the IPostEntityProcessorService, since the collision detection and resolution should be done after any movement.

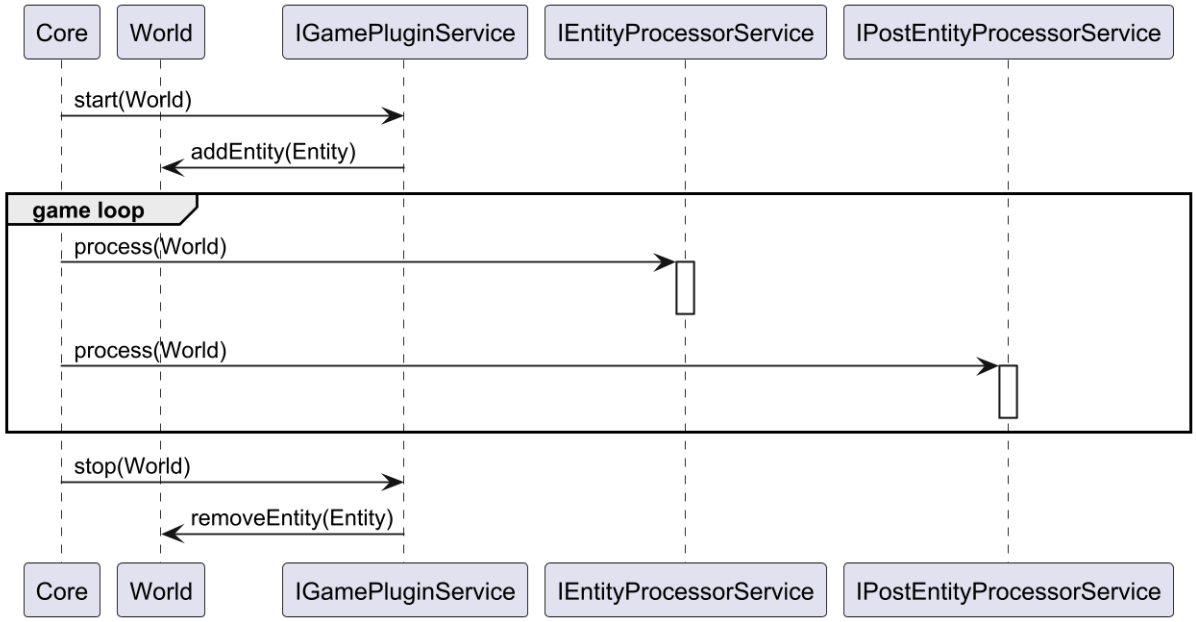

Figure 3: Sequence Diagram showing how Core communicates with the contracts.

On Figure 3 you can see how the three contracts play together in the system. At the very beginning, Core calls the start method on all IGamePluginService implementations with the World as argument. The services would then add their respective entities to the world. Then the main loop start, where the IEntityProcessorService implementations are called first, followed by the IPostEntityProcessorService implementations. Both have a method, process, which takes the world as an argument. Finally, the stop method is invoked, and any remaining entities are removed from the world. The World would contain a reference to all the entities which the processor would access and manipulate as need be.

4. Design

A proper design is crucial in a component-based system. This section will dive into what choices have been made to fulfill all relevant requirements. Complying with requirement NF04, the system is designed in a data-oriented way by utilizing Entity-Component-System architecture traits. Entities serve strictly as containers of various components. There exists zero logic on the individual entities, apart from their creation, which is minimal. Components contain data in all shapes and sizes. This could be everything from just a singular boolean flag, to large meshes and materials.

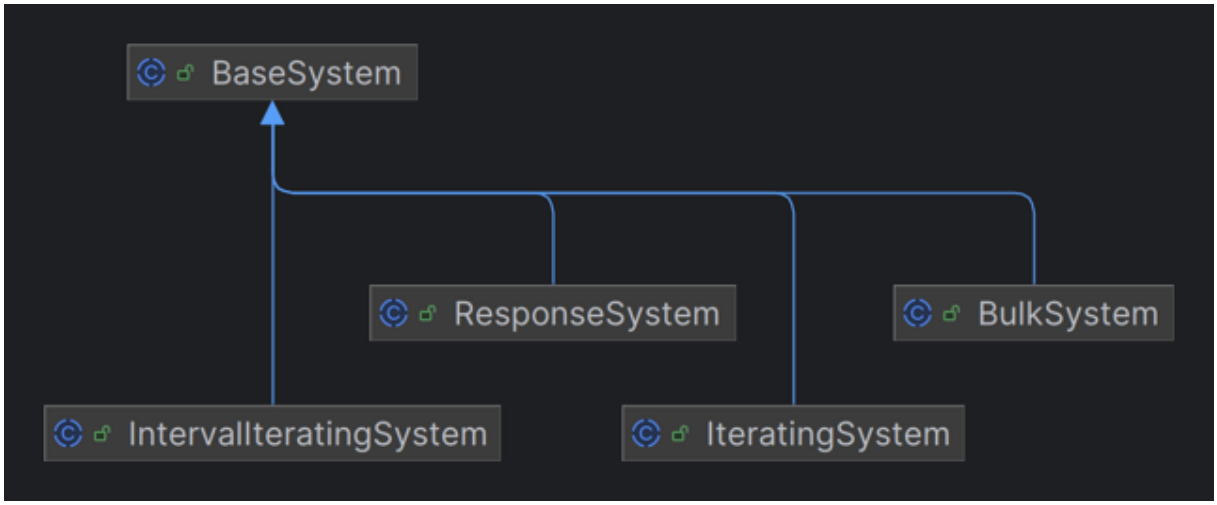

Figure 4: Four sub systems specializing in various patterns.

Systems query the world for entities that contain a collection of components the system is interested in. This is where all the logic is implemented. Different systems have different running characteristics. To accommodate this, four different specialized systems (see Figure 4) have been created, on top of the BaseSystem. The most common is the IteratingSystem. As the name suggests, it iterates through the list of entities matching its signature, one by one. In contrast to the BulkSystem that hands over the entire list of entities all at once, which in some cases is beneficial, for example in a collision detection system where comparison is necessary between entities. The IntervalIteratingSystem is an IteratingSystem on a configurable timer, only firing the update method once per interval. Finally, the ResponseSystem is a way to tell the game that I'm just here to listen to events.

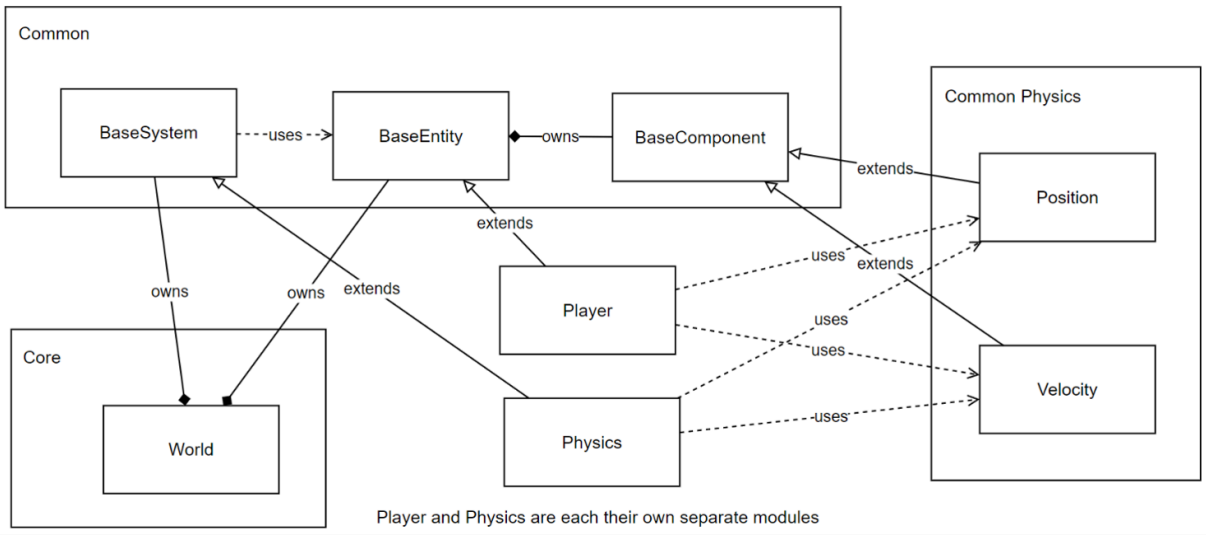

Figure 5: ECS Diagram.

Figure 5 illustrates how the components relate to each other. Notably, there is no direct connection between the Player and Physics modules, despite the Player being affected by physical calculations. This decoupling is achieved through shared contracts. The Player extends the BaseEntity class, which resides in the World. The Physics module is a BaseSystem within that same World. To process entities, the World queries the Physics system through the BaseSystem abstraction to determine what components it expects. Having all the required components serves as the pre-condition for any entity to be processed by that system. The World then filters the entities and passes a list of matching ones to the Physics system. The Position and Velocity components are declared in a common module, that both Physics and Player know about and rely on. This allows the Physics system to manipulate the Player's position without having any dependency on the Player itself. The system's post-condition is fulfilled once it has mutated the relevant component data of the entities. This same principle is applied to all the different systems throughout the game. Another feature of the systems is the priority value. This allows certain systems to run before or after other systems. This is necessary in certain physics calculations where acceleration is applied before velocity. This is also used to ensure that rendering is always the last thing each frame, to make sure that the user is seeing the newest state of the game.

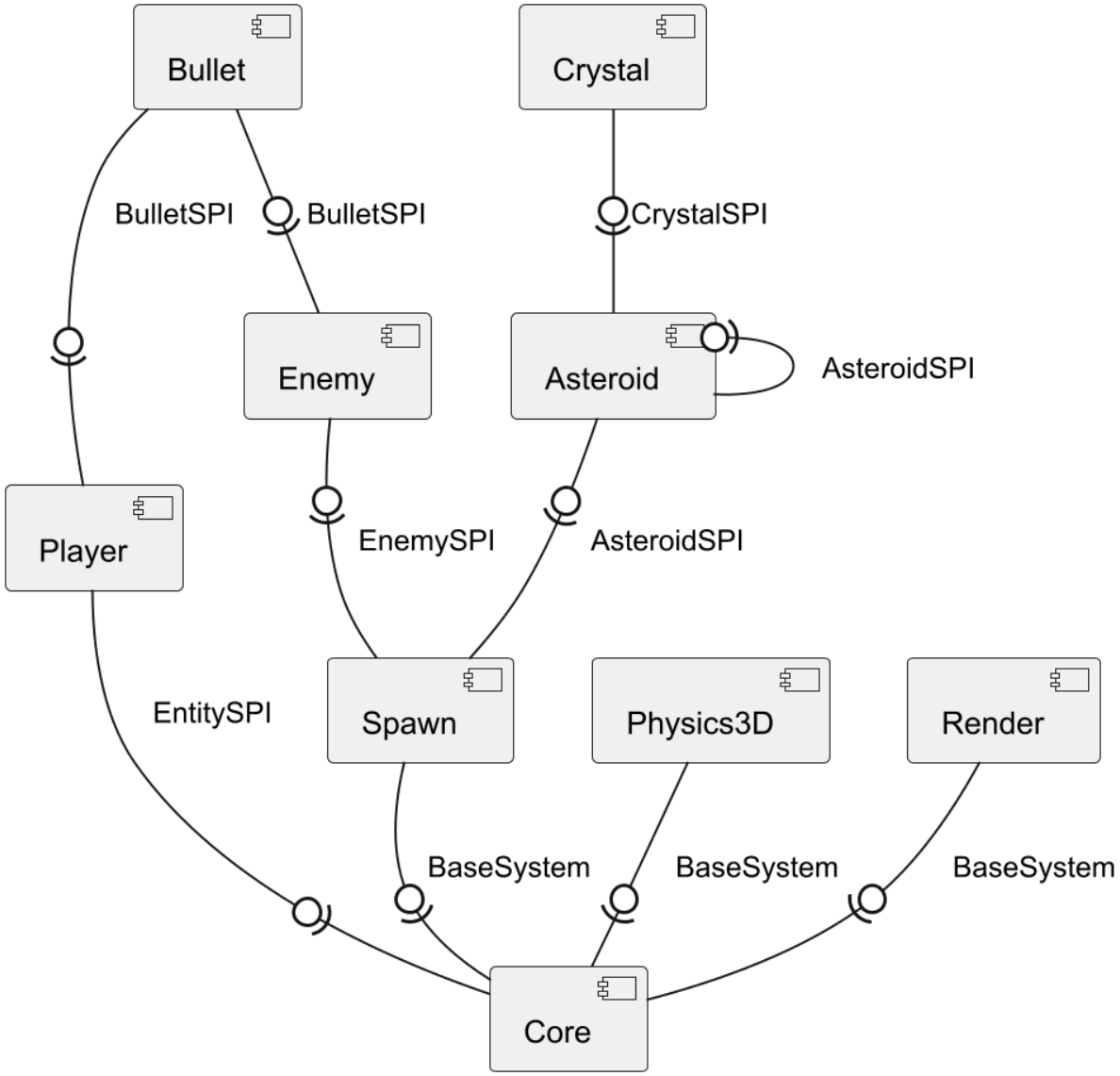

Figure 6: Partial Component Diagram.

Figure 6 shows the modular provides-requires flow of the system. The Player module provides an EntitySPI interface implementation that the Core module uses. Likewise, all the system modules, some of which are shown on the diagram, are providing BaseSystem realizations which the Core module also uses. Enemies and asteroids are controlled by a Spawn system. This requires an Enemy- and AsteroidSPI declared in their own common modules, CommonEnemy and CommonAsteroid respectively. Both Player and Enemy use the BulletSPI interface when shooting. The Asteroid module contains the system spawning crystals on hit, hence the requirement to the CrystalSPI. The discovery between modules is utilizing the Java ServiceLoader pattern, along with internal Spring Dependency Injection. This allows the Core to discover and inject implementations at runtime without any hardcoded dependencies. This satisfies requirements NF01, NF02, NF03, NF07, and NF08. The Scoring Service provides endpoints for incrementing and retrieving the score through a REST-based microservice, satisfying requirement NF05.

5. Implementation

In the following section, the implementation of the system will be documented.

5.1 Modular Encapsulation & Dependencies

To ensure strong encapsulation and reliable dependencies, the Java Platform Module System (JPMS) is used to dictate module accessibility and visibility across the entire system. This is required by NF01, NF02, NF03, and NF07.

```
module Asteroid {
    requires Common;
    requires CommonAsteroid;
    requires CommonCollision;
    requires CommonCrystal;
    requires CommonEnemy;
    requires CommonLifetime;
    requires CommonPhysics3D;
    requires CommonRender;
    requires CommonSpawn;
    requires spring.beans;
    requires spring.context;

    opens io.asteroidsjaylib.asteroid to spring.core, spring.context, spring.beans;

    uses AsteroidSPI;
    uses CrystalSPI;

    provides AsteroidSPI with AsteroidProvider;
    provides BaseSystem with AsteroidCollisionResponseSystem;
}
```

Code Snippet 1: Asteroid module-info file. Imports are excluded.

Code Snippet 1 shows the module-info.java file for the Asteroid module. This module contains the AsteroidProvider, AsteroidEntity, and AsteroidCollisionResponseSystem classes.

The module requires a variety of Common modules. These requires directives tell what other modules must be present for this module to compile and run. This can be seen when compiling the game, where the Common module is the first to get compiled, followed by the other Common modules, and finally the specific implementations.

The opens keyword is used to allow other modules to use reflection within the Asteroid module. Here some sub-packages of the Spring framework are allowed such access. This is required because of the usage of Spring's ApplicationEventPublisher and @EventListener used for event-driven communication throughout the game.

The uses directive signals that this module will act as a consumer, allowing the ServiceLoader to discover external implementations of the specified types. This is here to allow the collision response class to spawn new asteroids and crystals on hit, through the service provider interface implementations.

Finally, the provides … with syntax specifies that this module acts as a service provider. It exposes its internal AsteroidProvider and AsteroidCollisionResponseSystem to any module that uses the AsteroidSPI and BaseSystem types, respectively.

Ultimately, this configuration guarantees that no other module can access the internal implementations of the Asteroid module. By restricting access to everything except the Spring

framework and the explicit SPIs, the system achieves strong encapsulation and reliable dependencies.

```
import io.asteroidsjaylib.common.ecs.BaseSystem;
import io.asteroidsjaylib.common.ecs.EntitySpi;

module Core {
    requires Common;
    requires io.github.electronstudio.jaylib.ffm;
    requires spring.beans;
    requires spring.context;

    opens io.asteroidsjaylib to spring.core, spring.beans, spring.context;

    uses BaseSystem;
    uses EntitySPI;
}
```

Code Snippet 2: Core module-info file.

Code Snippet 2 shows the module-info.java file for the Core module. Here it is worth noting that it only requires the standard common module, and nothing else. It also declares that it uses the BaseSystem class implementations, as well as any EntitySPIs, like the Player. This allows the Core to discover and use the implementing modules through these types, without any dependency on them at compile-time.

5.2 Component Registration & Access

To successfully integrate the decoupled modules at runtime, the system uses a hybrid discovery and injection pattern utilizing both the Java ServiceLoader and the Spring Framework. This satisfies requirements NF01, NF02, NF03, and NF08.

```
@Configuration
@ComponentScan(basePackages = "io.asteroidsjaylib")
public class AppConfig {

    @Autowired
    private ConfigurableListableBeanFactory beanFactory;

    @Bean
    public List<BaseSystem> baseSystems() {
        List<BaseSystem> systems = new ArrayList<>();
        for (BaseSystem system : ServiceLoader.load(BaseSystem.class)) {

            String beanName = system.getClass().getName();

            beanFactory.registerSingleton(beanName, system);
            beanFactory.autowireBean(system);
            beanFactory.initializeBean(system, beanName);
            systems.add(system);
        }
        return systems;
    }

    @Bean
    public List<EntitySPI> entitySpis() {
        /* Similar logic as above */
    }

}
```

Code Snippet 3: Appconfig class

Code Snippet 3 shows the AppConfig class, which is annotated with @Configuration. This class acts as the bridge between JPMS module discovery and the Spring Inversion of Control (IoC) container. Because the external modules are completely decoupled from the Core, Spring cannot scan their packages directly. To circumvent this, the configuration autowires a ConfigurableListableBeanFactory. Within the bean producer methods, the Java ServiceLoader is used to dynamically discover all provided implementations of BaseSystem and EntitySPI on the module path. Once discovered, these instances are registered with the Spring factory as singletons based on their class names. Spring then autowires and initializes these dynamically loaded classes, bringing them fully under the management of the Spring context.

```
static void main() {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
    Game game = context.getBean(Game.class);
    game.start();
    context.close();
}

@Autowired
public Game(List<BaseSystem> systems, List<EntitySPI> entitySPIS) {
    this.systems = systems;
    this.entitySPIS = entitySPIS;
}
```

Code Snippet 4: Main method entrypoint and Game constructor.

With the components registered, the application entry point (see Code Snippet 4) initializes the AnnotationConfigApplicationContext using AppConfig.class. It then requests the primary Game bean. The Game class, annotated with @Component, uses constructor injection, explicitly requiring a List and a List as its arguments. Because the AppConfig registered all discovered plugins into the application context, Spring satisfies these dependencies. This now allows the Game to add the systems and entities to the World, without having to use the ServiceLoader directly.

5.3 ECS Component Models

To fulfill the data-oriented design requirement (NF04), the implementation separates state from behavior. The following sections explain how components are registered to entities, how systems query these entities based on components, and how the core optimizes this access at runtime.

Component Registration

Within the ECS architecture, entities serve as data containers. They are constructed by dynamically attaching the relevant data components during instantiation.

```
public class AsteroidEntity extends BaseEntity {
    public AsteroidEntity(Vector3D startPosition, Vector3D startVelocity, /* ... */)
{
        this.add(new Position().vector(startPosition));
        this.add(new Velocity().vector(startVelocity));
        // Additional component registration...
    }
}
```

Code Snippet 5: Component registration within the AsteroidEntity constructor.

As illustrated in Code Snippet 5, an AsteroidEntity is populated with Position and Velocity components. The add() method registers these components internally within the BaseEntity, making the data available for system queries.

System Logic and Component Access

Once components are registered to entities within the World, the specialized systems execute their domain logic by accessing this data.

```
public class VelocitySystem extends IteratingSystem {

    @Override
    public void start(IWorld world) {
        this.priority(22);
    }

    @Override
    public List<Class<? extends BaseComponent>> signature() {
        return List.of(Position.class, Velocity.class);
    }

    @Override
    public void update(IWorld world, BaseEntity entity, float deltaTime) {
        Position position = entity.get(Position.class);
        Velocity velocity = entity.get(Velocity.class);

        assert position != null;
        assert velocity != null;

        position.vector().addScaled(velocity.vector(), deltaTime);
    }
}
```

Code Snippet 6: The VelocitySystem defining its signature and mutation loop.

Code Snippet 6 shows the VelocitySystem, which extends the IteratingSystem class. The signature() method defines the system's pre-condition: It requires entities to have both Position and Velocity

components. The World uses this signature to query for matching entities. Because it is an iterating system, the update() method processes these entities sequentially, fetching the specific components via entity.get() and mutating the data by applying the velocity vector scaled by deltaTime.

Query Optimization and Caching

Continuously querying all entities against every system signature every frame introduces computational overhead ($O(N \times M)$ complexity). To ensure the game loop remains performant, the World class implements an optimized caching mechanism.

```
private final Map<BaseSystem, List<BaseEntity>> systemEntityCache = new HashMap<>();

private void updateCacheIfNeeded() {
    boolean entitiesChanged = entities.removeIf(BaseEntity::removed);

    if (!entitiesToAdd.isEmpty()){
        entities.addAll(entitiesToAdd);
        entitiesToAdd.clear();
        entitiesChanged = true;
    }

    if (entitiesChanged || systemEntityCache.isEmpty()){
        for (BaseSystem system : systems){
            List<Class<? extends BaseComponent>> signature = system.signature();

            List<BaseEntity> matching = systemEntityCache.computeIfAbsent(system, _ -
> new ArrayList<>());
            matching.clear();

            if (signature != null && !signature.isEmpty()){
                for (BaseEntity entity : entities){
                    if (entity.hasAll(signature)) matching.add(entity);
                }
            }
            systemEntityCache.put(system, matching);
        }
    }
}
```

Code Snippet 7: Entity cache recalculation logic within the World object.

Code Snippet 7 shows the updateCacheIfNeeded method, which manages the Map<BaseSystem, List<BaseEntity>>. Rather than querying components on every frame, the cache is only invalidated and recalculated when a change occurs. This happens when entities are marked for removal or new entities are added to the world. If a change is detected, the method iterates through each registered system, compares the active entities against the system's signature(), and repopulates the cached lists. This caching strategy prevents the world from querying all entities every frame, ensuring that the system updates remain highly efficient.

5.4 Microservice Integration

To fulfill the external service requirement (NF05), the game's scoring mechanic is decoupled from the main client and managed by an external microservice. This ensures the score state can be persisted and monitored independently of the game process.

The Standalone Scoring Service

The backend is implemented as a Spring Boot application exposing a RESTful API.

```
@SpringBootApplication
@RestController
public class ScoringService {

    private int score = 0;

    public static void main(String[] args) {
        SpringApplication.run(ScoringService.class, args);
    }

    @GetMapping("/score")
    public int score() {
        return score;
    }

    @PostMapping("/score")
    public void score(@RequestParam(value = "increment") int increment) {
        score += increment;
    }
}
```

Code Snippet 8: The standalone Spring Boot REST microservice.

As seen in Code Snippet 8, the service maintains the accumulated score state and exposes HTTP GET and POST endpoints to retrieve and increment the value.

The Client-Side Event Listener

When a crystal is collected, a ScoreEvent is published to the event bus.

```
public class ScoreSystem extends ResponseSystem {

    private final String scoringServiceUrl = "http://localhost:8080/score";
    private final RestTemplate restTemplate = new RestTemplate();

    @EventListener
    private void handleScoreIncrement(ScoreEvent scoreEvent){
        String url = scoringServiceUrl + "?increment=" + scoreEvent.increment;

        // Execute asynchronously to prevent blocking the game loop
        CompletableFuture.runAsync(() -> {
            try {
                restTemplate.postForLocation(url, null);
            } catch (RestClientException e) {
                System.out.println("Score Service not responding: " +
e.getMessage());
            }
        });
    }
}
```

Code Snippet 9: The client-side ScoreSystem utilizing asynchronous HTTP requests.

Code Snippet 9 illustrates the ScoreSystem, which listens for these events. A critical feature of this implementation is the use of the runAsync method. Because network latency is highly unpredictable, executing synchronous HTTP requests could freeze the main game loop and ruin the user experience. Wrapping the logic inside a runAsync call offloads the network I/O to a separate worker thread, allowing it to complete independently. Furthermore, to ensure graceful degradation, the postForLocation call is wrapped in a try-catch block. If the external microservice is offline or unresponsive, the system safely catches the RestClientException, preventing a fatal crash and allowing the game to proceed normally.

5.5 Robustness & Verification

To ensure reliable system logic, unit testing is implemented. By isolating individual components, their behaviour can be tested without requiring the full game to be running.

Isolated Unit Testing with Mockito

To verify system logic without the full Raylib graphical engine or the Spring IoC container, external dependencies are simulated using the Mockito testing framework [4].

```
@ExtendWith(MockitoExtension.class)
class LifetimeSystemTest {

    private LifetimeSystem lifetimeSystem;

    @Mock private IWorld mockWorld;
    @Mock private BaseEntity mockEntity;
    @Mock private ITimeProvider mockTimeProvider;

    private Lifetime lifetime;

    @BeforeEach
    void setUp() {
        lifetimeSystem = new LifetimeSystem();
        lifetimeSystem.timeProvider = mockTimeProvider;

        // Simulate an entity spawned at 10.0s with a 5.0s lifetime
        lifetime = new Lifetime(10.0f, 5.0f);
    }

    @Test
    void givenNotRunOut_WhenUpdate_ThenDontRemove() {
        when(mockEntity.get(Lifetime.class)).thenReturn(lifetime);
        when(mockTimeProvider.getTime()).thenReturn(14.0f); // 4.0s alive

        lifetimeSystem.update(mockWorld, mockEntity, 0.016f);

        verify(mockEntity).get(Lifetime.class);
        verifyNoMoreInteractions(mockEntity); // Ensure removed() was NOT called
    }

    @Test
    void givenHasRunOut_WhenUpdate_ThenRemovesEntity() {
        when(mockEntity.get(Lifetime.class)).thenReturn(lifetime);
        when(mockTimeProvider.getTime()).thenReturn(16.0f); // 6.0s alive

        lifetimeSystem.update(mockWorld, mockEntity, 0.016f);

        verify(mockEntity).removed(true); // Ensure it WAS flagged for removal
    }
}
```

Code Snippet 10: Unit tests verifying the LifetimeSystem logic using Mockito.

As shown in Code Snippet 10, the LifetimeSystemTest isolates the system under test by injecting mocked instances of IWorld, BaseEntity, and ITimeProvider. A critical architectural advantage of this approach is the deterministic control over the time. Rather than relying on a live game loop, the test stubs the ITimeProvider.getTime() method using the when().thenReturn() syntax. This artificially advances the simulation, allowing the tests to verify various conditions. The assertions (verify()) prove that the removed(true) post-condition is exclusively triggered when the entity's lifetime is exceeded. This guarantees that the logic remains robust and perfectly decoupled from the execution environment.

5.6 Rendering and Memory Optimizations

To hit the visual requirements (F07, F08) and get the most out of Raylib (NF06), the game uses a custom setup to handle Level of Detail (LOD) and memory management.

The Java binding Jaylib-ffm [5] is used to make the bridge between Java and Raylib.

Level of Detail (LOD)

Drawing highly detailed 3D models when they are far away is a massive waste of processing power, especially when there are hundreds of objects in the game at once. To fix this (F07), the game uses a custom LODSystem.

```
public class LODSystem extends IteratingSystem {
// ...

    @Override
    public void update(IWorld world, BaseEntity entity, float deltaTime) {
        Position position = entity.get(Position.class);
        Render3D render3D = entity.get(Render3D.class);

        float distance = position.vector().magnitude();

        for(Base3DShape shape : render3D.getActiveShapes()){
            shape.currentLodLevel = Math.min(
                shape.lodCount - 1,
                (int)(distance / (5000.0f / shape.lodCount))
            );
        }
    }
}
```

Code Snippet 11: Calculating the LOD level based on distance.

Instead of loading separate models for every detail level, the game uses the fact that a single GLB file can contain multiple different meshes. The LODSystem (see Code Snippet 11) calculates how far an entity is from the camera. Based on that distance, it figures out exactly which currentLodLevel to use.

When drawing, the RenderSystem uses that LOD level to shift the array index, making sure it only grabs the geometry and materials it actually needs. A strict rule here is that when making the various LOD models in Blender, the mesh count per LOD has to stay exactly the same so the math doesn't break. This setup means the engine draws way fewer polygons for distant objects, keeping the frame rate smooth.

FFM and Shaders

To make the game look better with lighting and reflections (F08), the rendering pipeline uses custom shaders. However, passing data from Java to Raylib can cause massive memory issues.

```
public static void setGlobalShaderValue(String uniformName, float[] values, int
uniformType) {

    try (Arena arena = Arena.ofConfined()){

        MemorySegment segment = arena.allocateFrom(ValueLayout.JAVA_FLOAT, values);

        for (var shader : shaderMap.values()){
            int loc = getShaderLocation(shader, uniformName);

            if (loc != -1){
                setShaderValue(shader, loc, segment, uniformType);
            }
        }
    } catch (Exception e){
        e.printStackTrace();
    }
}
```

Code Snippet 12: Using FFM to pass shader data directly to native memory.

Normally, creating Java objects every frame to pass this data would trigger the Java Garbage Collector, causing the game to stutter. To avoid this, the game uses the Java Foreign Function & Memory (FFM) API. By wrapping the allocation in a try (Arena arena = Arena.ofConfined()) block, the game writes data straight to native memory using MemorySegment. The memory cleans itself up when the block finishes, skipping the garbage collector completely. (See Code Snippet 12).

6. Test

6.1 Building and Running the Game

To build the game, the mvn install command is used. This compiles all the modules into separate .jar files and puts them into a mods-mvn folder.

The game can now be executed using either the ‘mvn exec:exec’ command, or by using the ‘java -p mods-mvn -m Core/io.asteroidsjaylib.Main’ command. The java command is the one to be used by the end users of the game, wrapped as an executeable, because it only requires the .jar files, and nothing else.

6.2 Requirement Validation

All functional requirements have been implemented. Their existence can be seen in the demo video here: https://youtu.be/oF3Cr7Afvww.

- The X-wing player spacecraft is controlled by the player.
- The Tie-fighter enemy crafts are shooting bullets at the player.
- The asteroids are deadly upon impact, as well as being the source of crystals.
- Bullets are fired by either the player or the enemy.
- Crystals can be collected for points.
- The game handles 3D physics, including position, velocity, acceleration, and rotation.
- The rendering is capable of showing various meshes with various materials, including LOD support.
- The game uses shaders to make the bullets glow and mimic realistic lighting overall.

All of the non-functional requirements are also implemented. The video demonstrates the Moddable requirement by removing various .jar files and still having a functional game, just now with fewer features. The rest of the non-functional requirements are documented in the Implementation section above.

Finally, as detailed in Section 5.5, unit testing (NF09) was used to verify the internal logic of the systems. By using Mockito, the behaviour of the game was tested in isolation without needing to boot up the entire Raylib engine.

7. Discussion

The following section will discuss how well the game solved the main problem of modularity as well as meeting the requirements.

7.1 Solving the Main Problem

Using the Java Platform Module System proved to be an excellent way to allow decoupling and swapping modules in the game. The ServiceLoader API was very easy to utilize, to find implementations matching the common interfaces. The architecture allowed for decoupled and asynchronous development, since dependencies were few.

The Spring IoC Container was a bit of a mouthful at first, but it quickly proved its worth, by making the Game completely decoupled from the ServiceLoader in a very clean way. Just managing a single configuration file made sure that the critical use of the ServiceLoader was not being buried in the game logic.

One critique of this architecture would be that the encapsulation of JPMS and the nature of Spring were clashing a bit, because Spring relies heavily on the Reflection API, while JPMS forbids it. This was solved by using the 'opens … to …' declarative, but it felt wrong every time.

Overall it was a pleasure to develop a modular program in this way, and the overhead proved its worth quickly.

7.2 Meeting the Requirements

All of the set requirements were implemented as documented in the previous test section.

Using the ECS architecture made it very easy to add new features and entities to the game, which greatly increased development velocity.

The hardest feature to implement was definitely handling rotation in 3D, which required complex (literally) math in the form of quaternions. Luckily this is a well-documented topic in 3D game programming [6].

The Score Service Spring Boot Application requirement felt very out of place, but worked out pretty well.

8. Conclusion

This report documented the development of the Asteroids3D game, a modern take on the classic Asteroids arcade game built using JPMS, Spring, ECS and Raylib.

The primary goal of this project was to develop a modular game where .jar files could be added or removed without the need of recompiling the core game. This was accomplished using Java modules and the ServiceLoader API, which in turn gave the following advantages:

- Strict Encapsulation: The internal implementation was hidden behind module boundaries.
- Reliable Dependencies: All communication between modules was done through stable contracts.
- Low Coupling: Modules only depended on common contracts, and not each other, allowing modules to be implemented independently.
- High Cohesion: Modules were very specific in their domain, and god classes were avoided.

Future work includes adding more gameplay features and advanced enemies with various attack patterns. Some of the more technical classes, like the ShaderManager, could also do with a refactor to increase further development velocity even more.

9. References

[1] raysan5, “raylib | A simple and easy-to-use library to enjoy videogames programming.” Accessed: Mar. 05, 2026. [Online]. Available: https://www.raylib.com/

[2] Oracle, “ServiceLoader.” Accessed: Feb. 25, 2026. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/ServiceLoader.html

[3] V. Tanzu, “Spring.” Accessed: Apr. 19, 2026. [Online]. Available: https://spring.io/

[4] S. Faber and friends, “Mockito framework site.” Accessed: Apr. 21, 2026. [Online]. Available: https://site.mockito.org/

[5] electronstudio, “jaylib-ffm.” Accessed: May 06, 2026. [Online]. Available: https://github.com/electronstudio/jaylib-ffm

[6] Jeremiah, “Understanding Quaternions.” Accessed: Apr. 11, 2026. [Online]. Available: https://www.3dgep.com/understanding-quaternions/